Introduction to Algorithms

Email: grad.saeed@gmail.com

Github: github.com/saeedgrad

Designing Algorithms

- Insertion sort uses the incremental method: for each element A[i], insert it into its proper place in the subarray A[1:i], having already sorted the subarray A[1:i-1].
- This section examines another design method, known as divide-and-conquer.
- Worst-case running time for divide-and-conquer is much less than that of insertion sort.
- Analyzing its running time is often straightforward.

The divide-and-conquer method

- Many useful algorithms are recursive in structure: to solve a given problem, they recurse (call themselves) one or more times to handle closely related subproblems.
- They break the problem into several subproblems that are similar to the original problem but smaller in size, solve the subproblems recursively, and then combine these solutions to create a solution to the original problem.
- Base case: if the problem is small enough-the base Case- you just solve it directly without recursing.

The divide-and-conquer method

- Recursive case:

- **Divide** the problem into one or more subproblems that are smaller instances of the same problem.

- **Conquer** the subproblems by solving them recursively.

- **Combine** the subproblem solutions to form a solution to the original problem.

The merge sort algorithm

- **Divide** the subarray A[p:r] to be sorted into two adjacent subarrays, each of half the size. To do so, compute the midpoint q of A[p:r] (taking the average of p and r), and divide A[p:r] into subarrays A[p:q] and A[q+1:r].

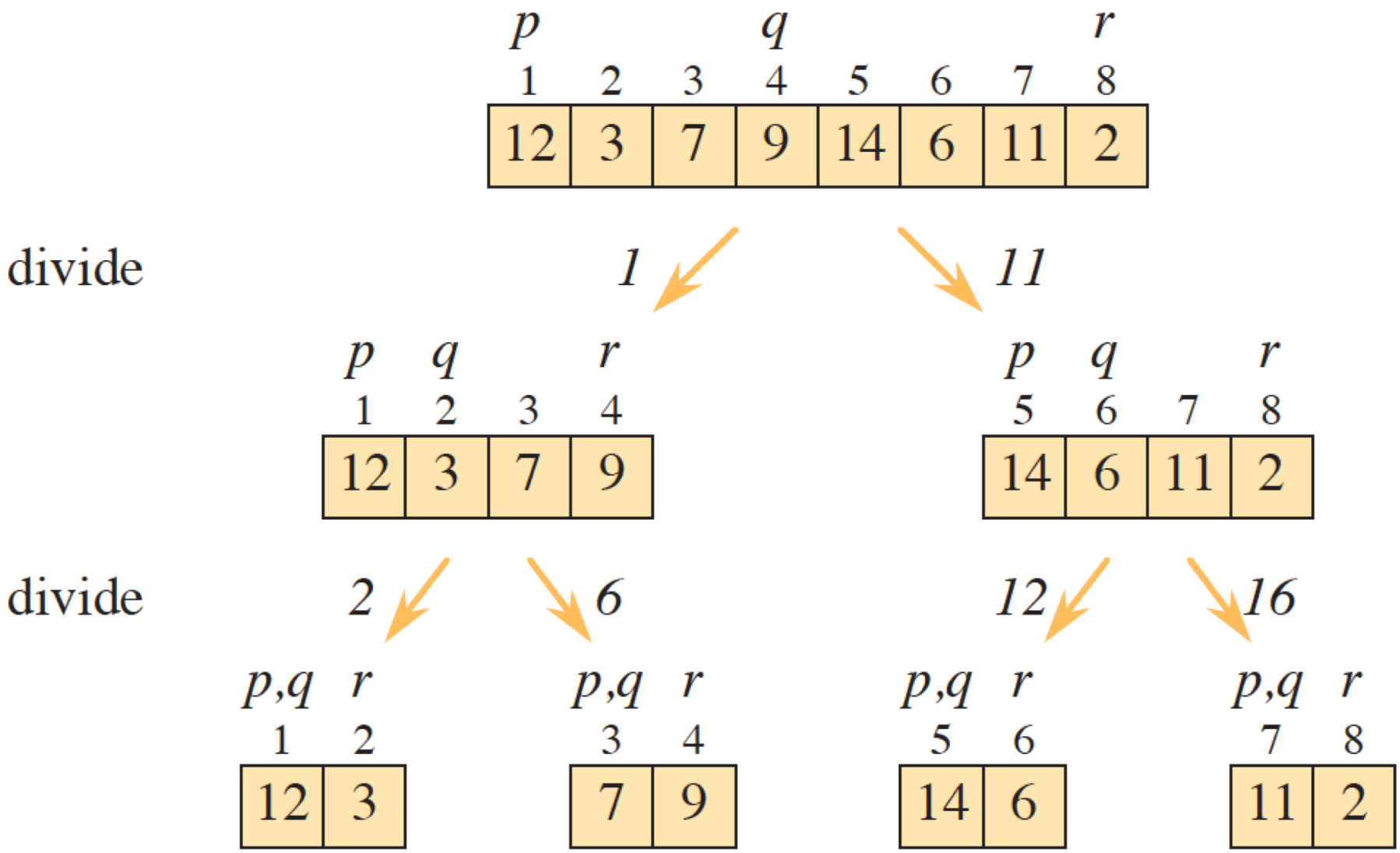
- **Conquer** by sorting each of the two subarrays A[p:q] and A[q+1:r] recursively using merge sort.

- **Combine** by merging the two sorted subarrays A[p:q] and A[q+1:r] back into A[p:r], producing the sorted answer.

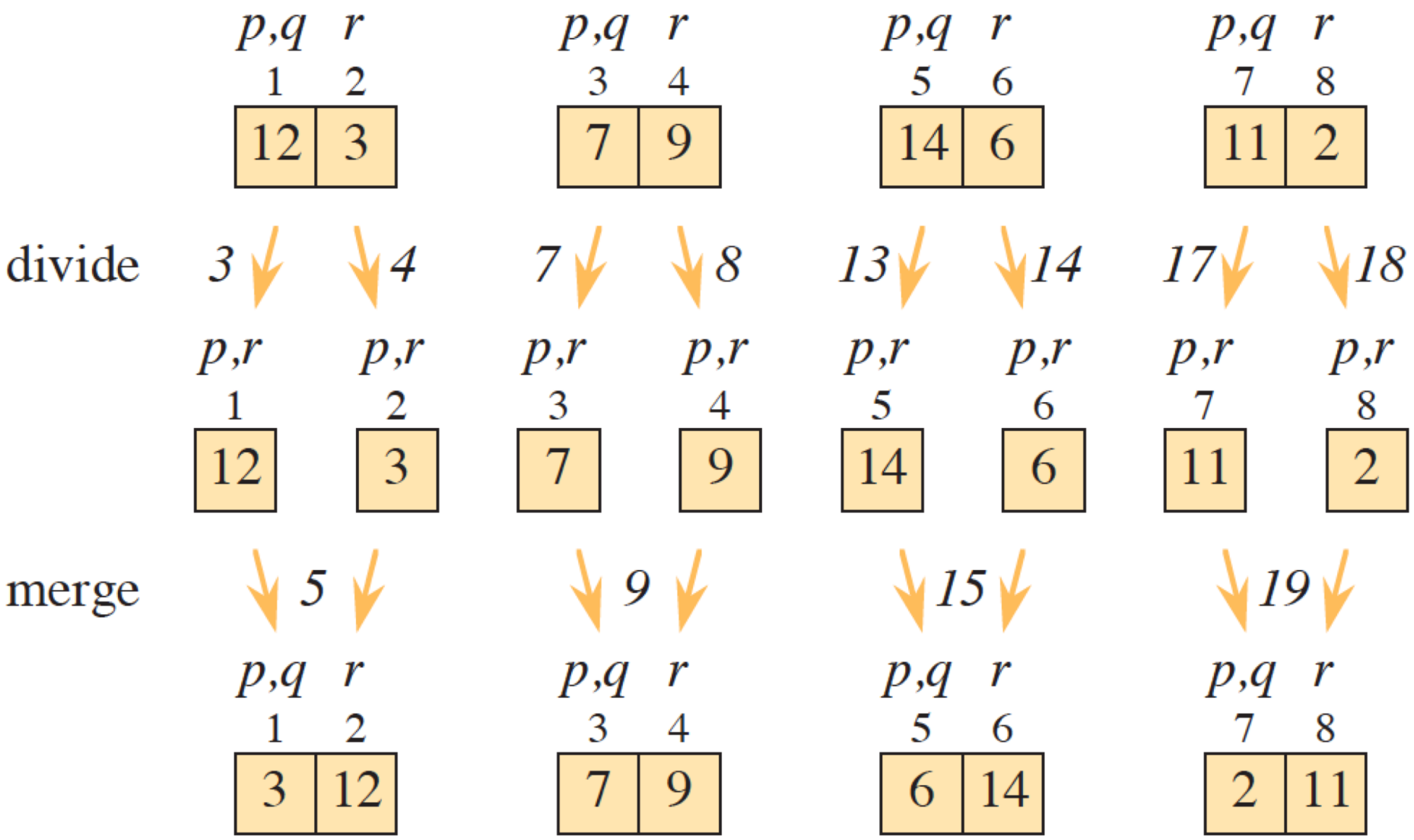
The procedure MERGE-SORT

```
MERGE-SORT(A, p, r)
1  if p ≥ r                                  // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                           // midpoint of A[p : r]
4  MERGE-SORT(A, p, q)                       // recursively sort A[p : q]
5  MERGE-SORT(A, q + 1, r)                   // recursively sort A[q + 1 : r]
6  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7  MERGE(A, p, q, r)
```

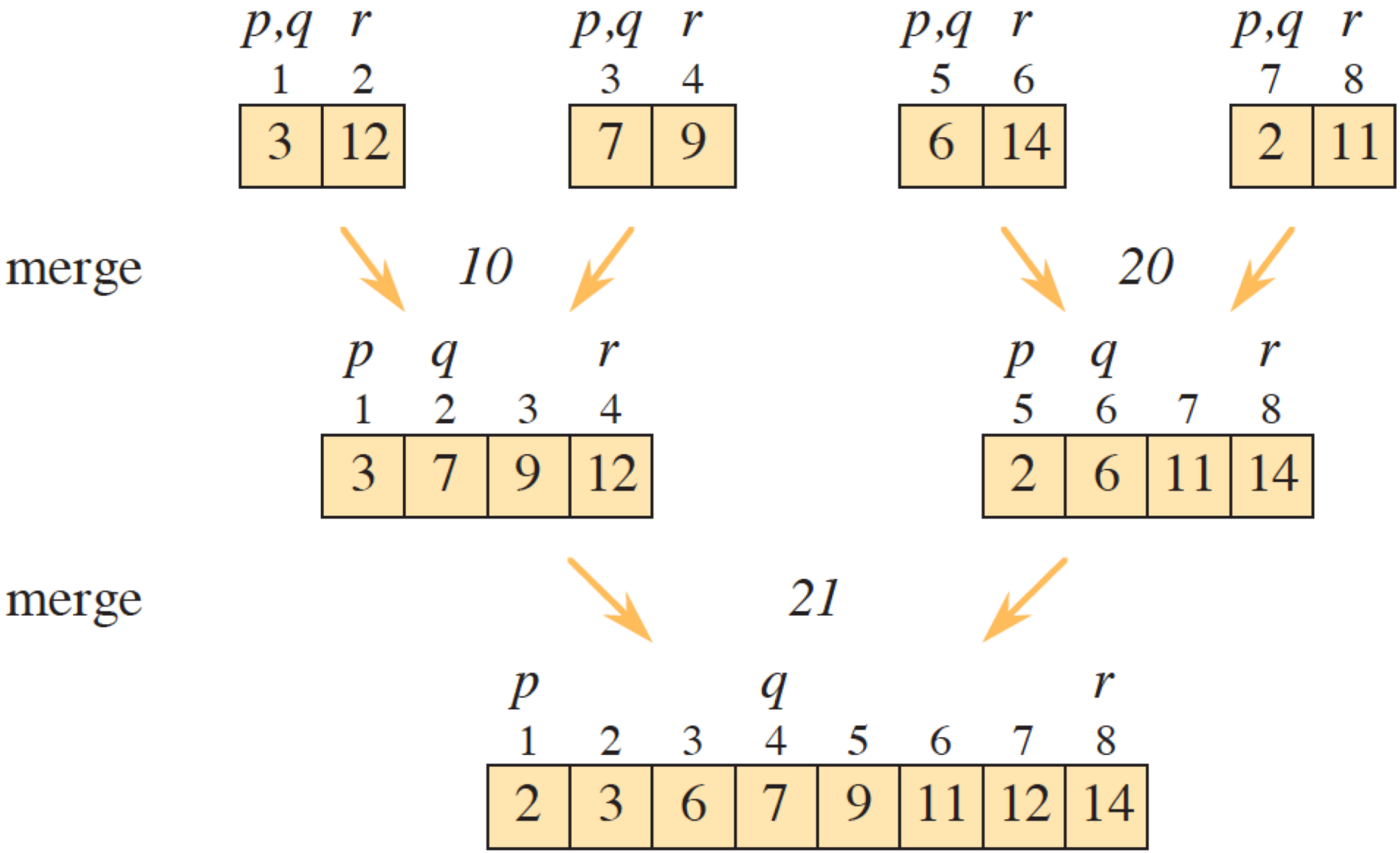
MERGE-SORT example



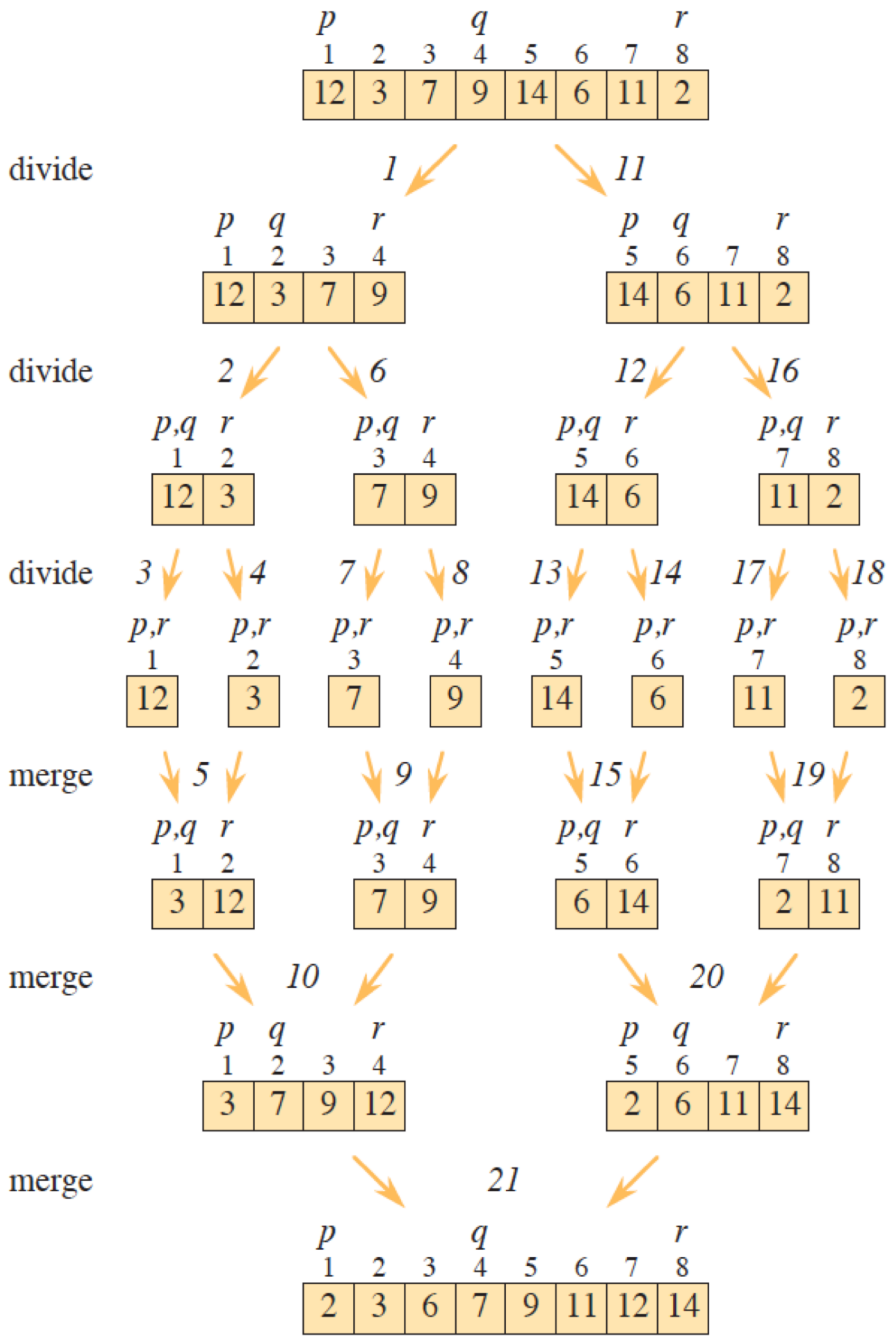
MERGE-SORT example cont'd



MERGE-SORT example cont'd



MERGE-SORT example cont'd



The merge algorithm

- The key operation of the merge sort algorithm occurs in the combine step, which merges two adjacent, sorted subarrays.
- The merge operation is performed by the auxiliary procedure $\text{MERGE}(A, p, q, r)$
- The procedure assumes that the adjacent subarrays A[p:q] and A[q+1:r] were already recursively sorted.
- It merges the two sorted subarrays to form a single sorted subarray that replaces the current subarray A[p:r].

$$p \leq q < r$$

The procedure MERGE

```
MERGE(A, p, q, r)
 1  n_L = q − p + 1          // length of A[p : q]
 2  n_R = r − q              // length of A[q + 1 : r]
 3  let L[0 : n_L − 1] and R[0 : n_R − 1] be new arrays
 4  for i = 0 to n_L − 1     // copy A[p : q] into L[0 : n_L − 1]
 5      L[i] = A[p + i]
 6  for j = 0 to n_R − 1     // copy A[q + 1 : r] into R[0 : n_R − 1]
 7      R[j] = A[q + j + 1]
 8  i = 0                    // i indexes the smallest remaining element in L
 9  j = 0                    // j indexes the smallest remaining element in R
10  k = p                    // k indexes the location in A to fill
```

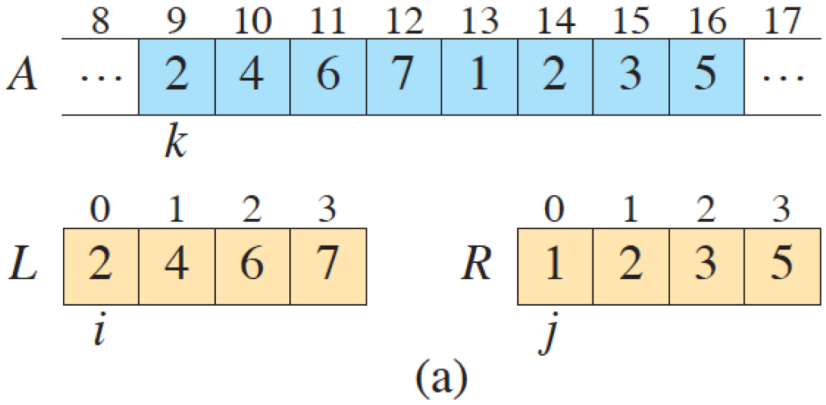
The procedure MERGE cont'd

```
11  // As long as each of the arrays L and R contains an unmerged element,
    //     copy the smallest unmerged element back into A[p : r].
12  while i < n_L and j < n_R
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
18      k = k + 1
```

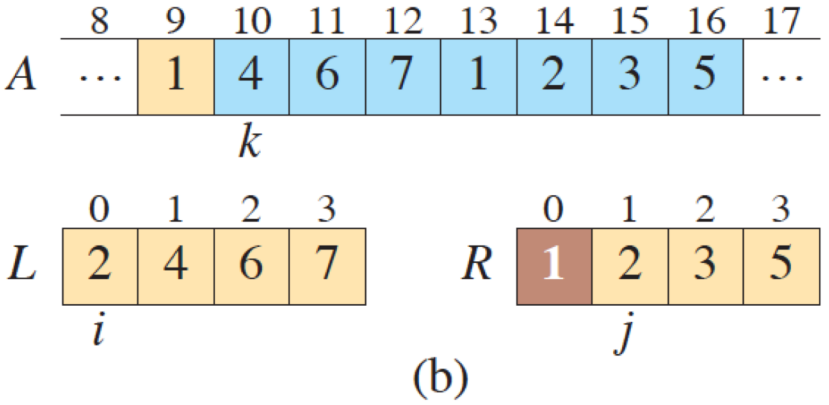
The procedure MERGE cont'd

```
19  // Having gone through one of L and R entirely, copy the
    //      remainder of the other to the end of A[p : r].
20  while i < n_L
21      A[k] = L[i]
22      i = i + 1
23      k = k + 1
24  while j < n_R
25      A[k] = R[j]
26      j = j + 1
27      k = k + 1
```

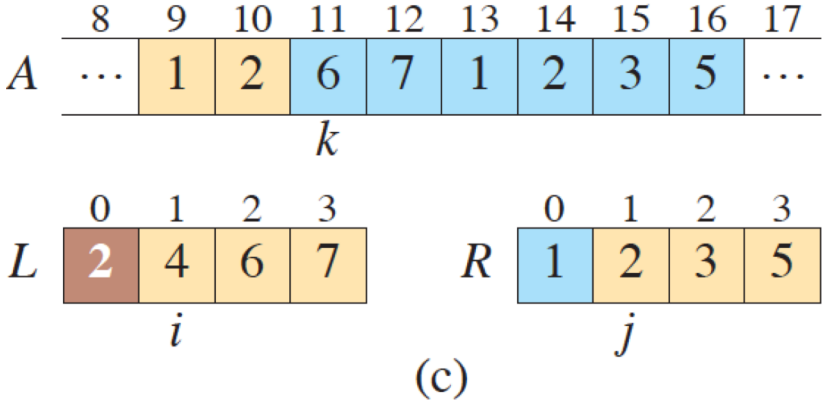
MERGE example



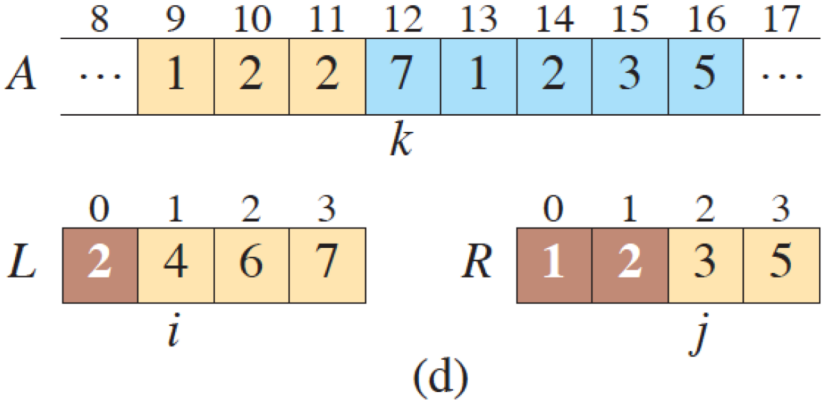
(a)



(b)

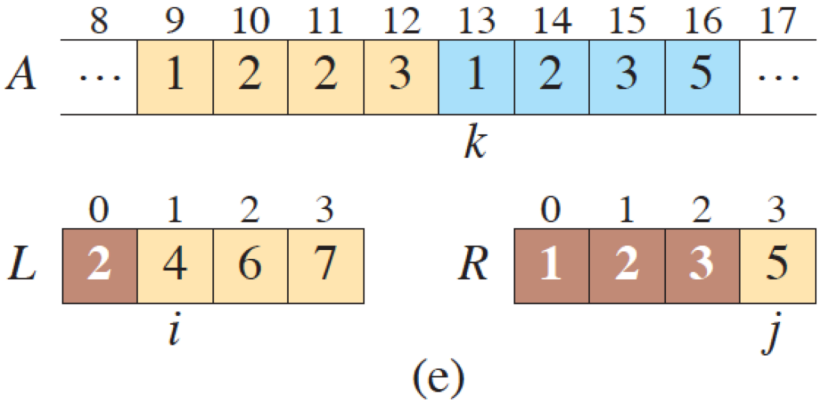


(c)

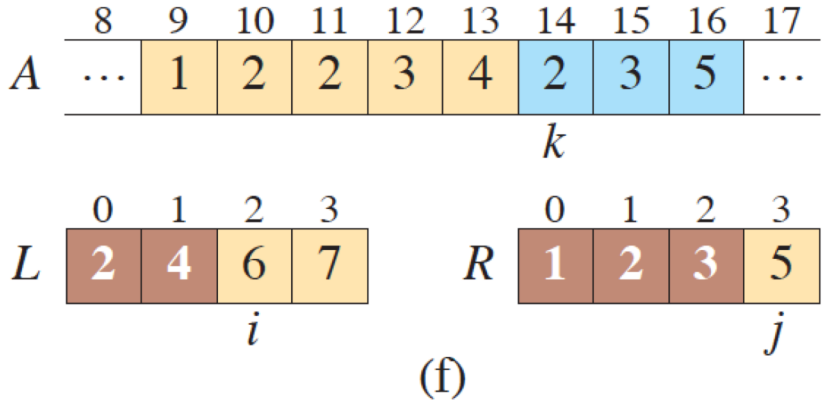


(d)

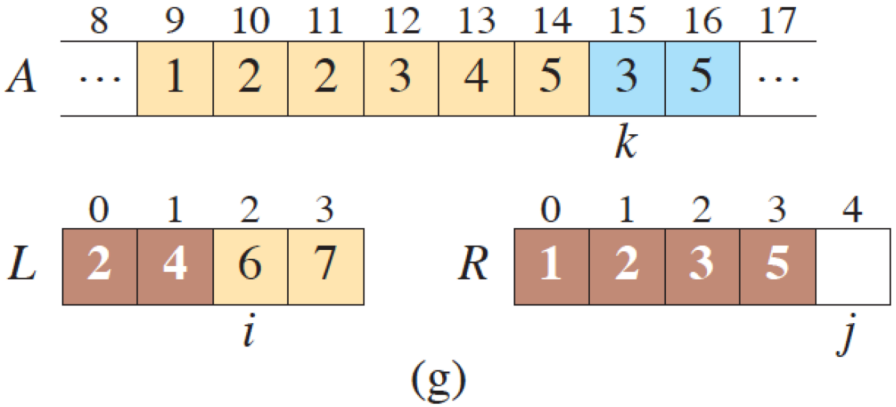
MERGE example cont'd



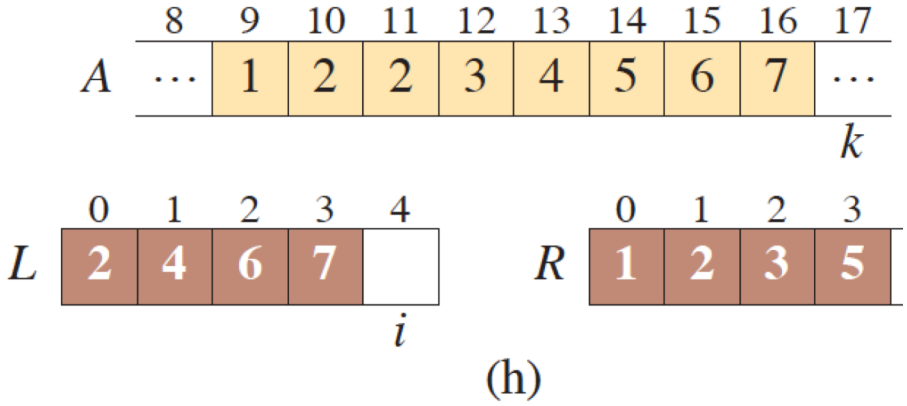
(e)



(f)



(g)



(h)

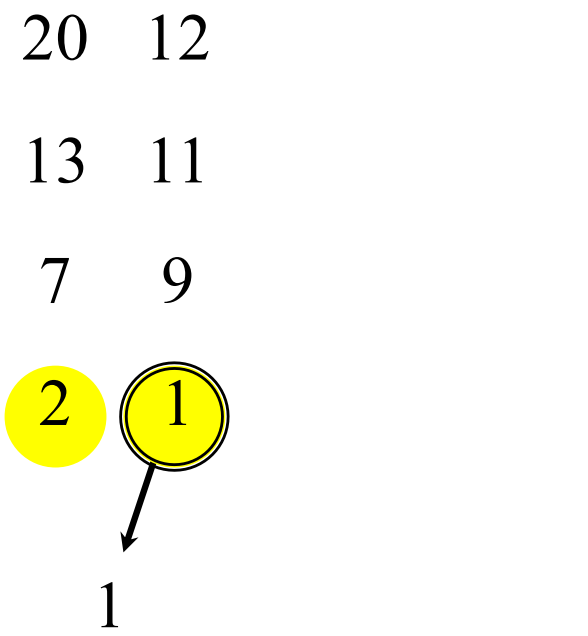
MERGE running time

To see that the MERGE procedure runs in $\Theta(n)$ time, where $n = r - p + 1$,[13] observe that each of lines 1–3 and 8–10 takes constant time, and the **for** loops of lines 4–7 take $\Theta(n_L + n_R) = \Theta(n)$ time.[14] To account for the three **while** loops of lines 12–18, 20–23, and 24–27, observe that each iteration of these loops copies exactly one value from L or R back into A and that every value is copied back into A exactly once. Therefore, these three loops together make a total of n iterations. Since each iteration of each of the three loops takes constant time, the total time spent in these three loops is $\Theta(n)$.

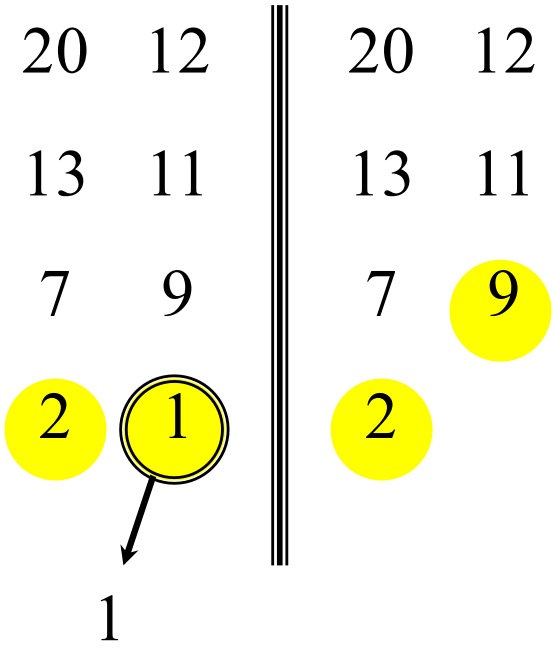
Merging two sorted arrays

20	12
13	11
7	9
2	1

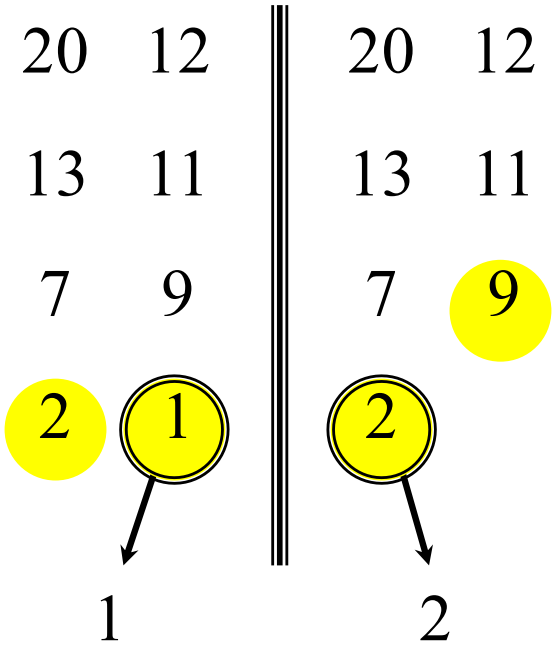
Merging two sorted arrays



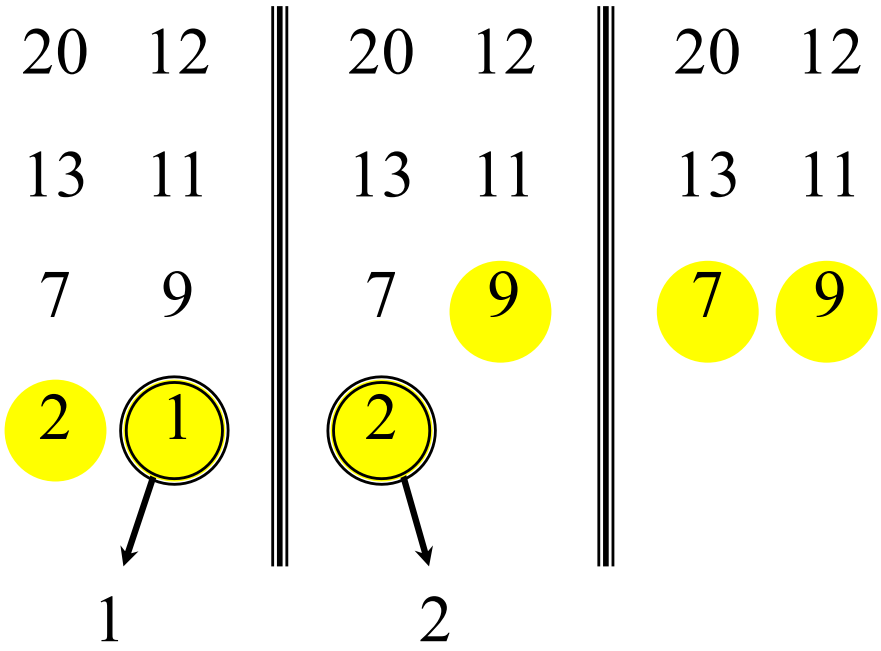
Merging two sorted arrays



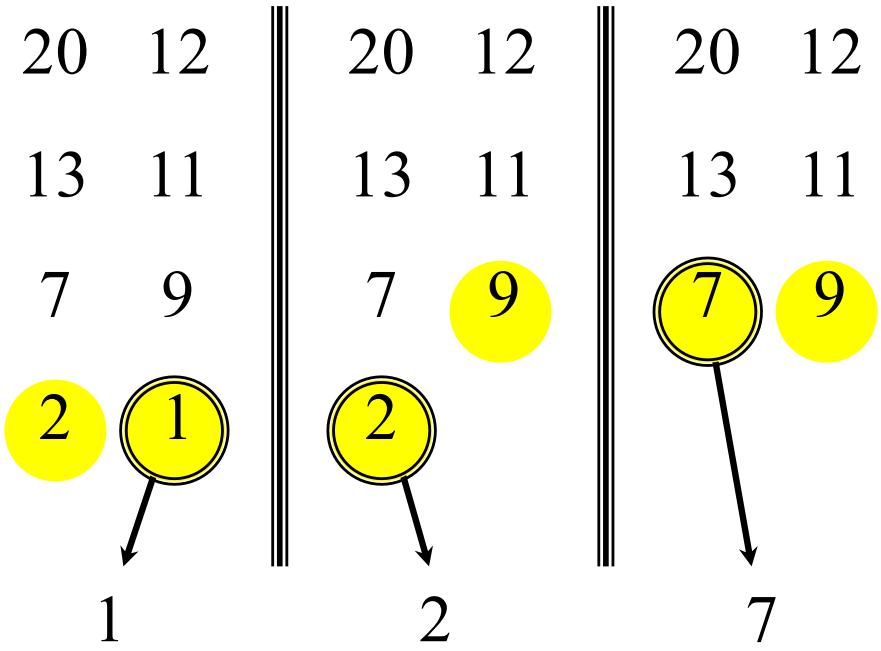
Merging two sorted arrays



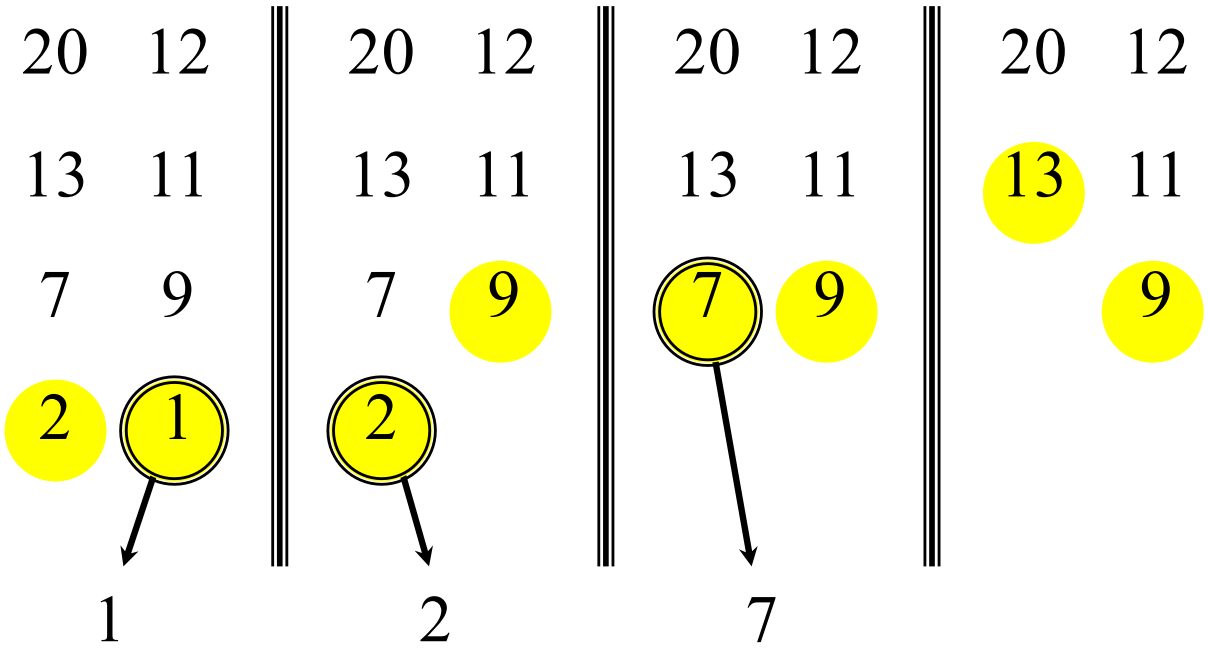
Merging two sorted arrays



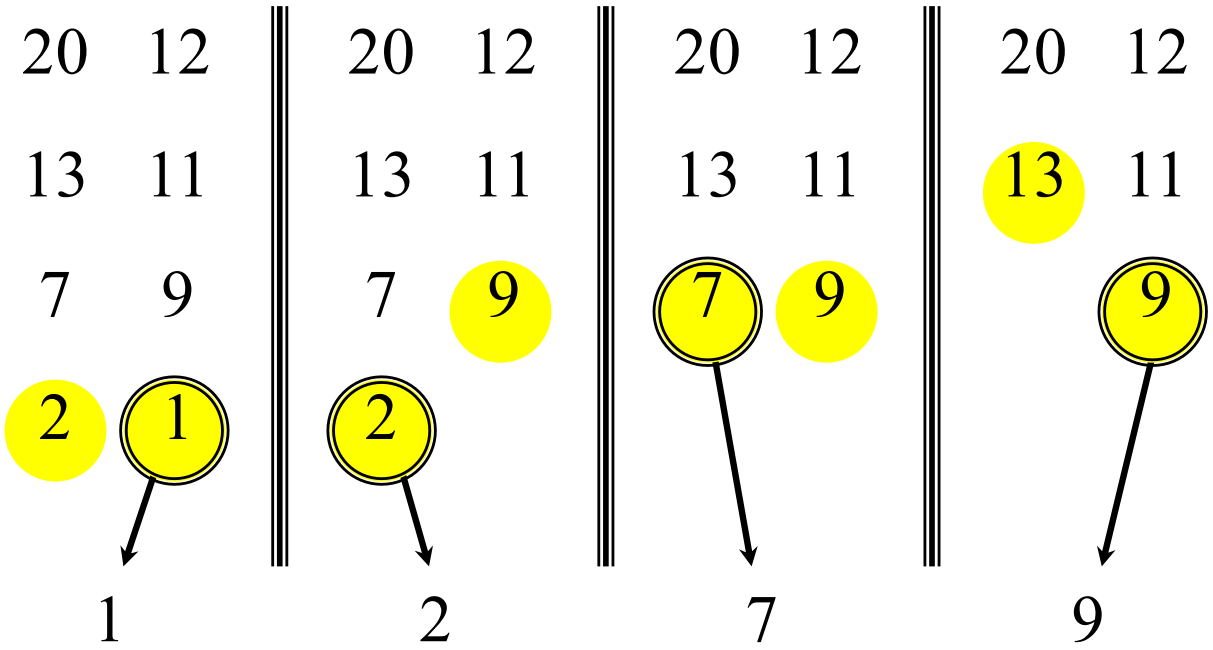
Merging two sorted arrays



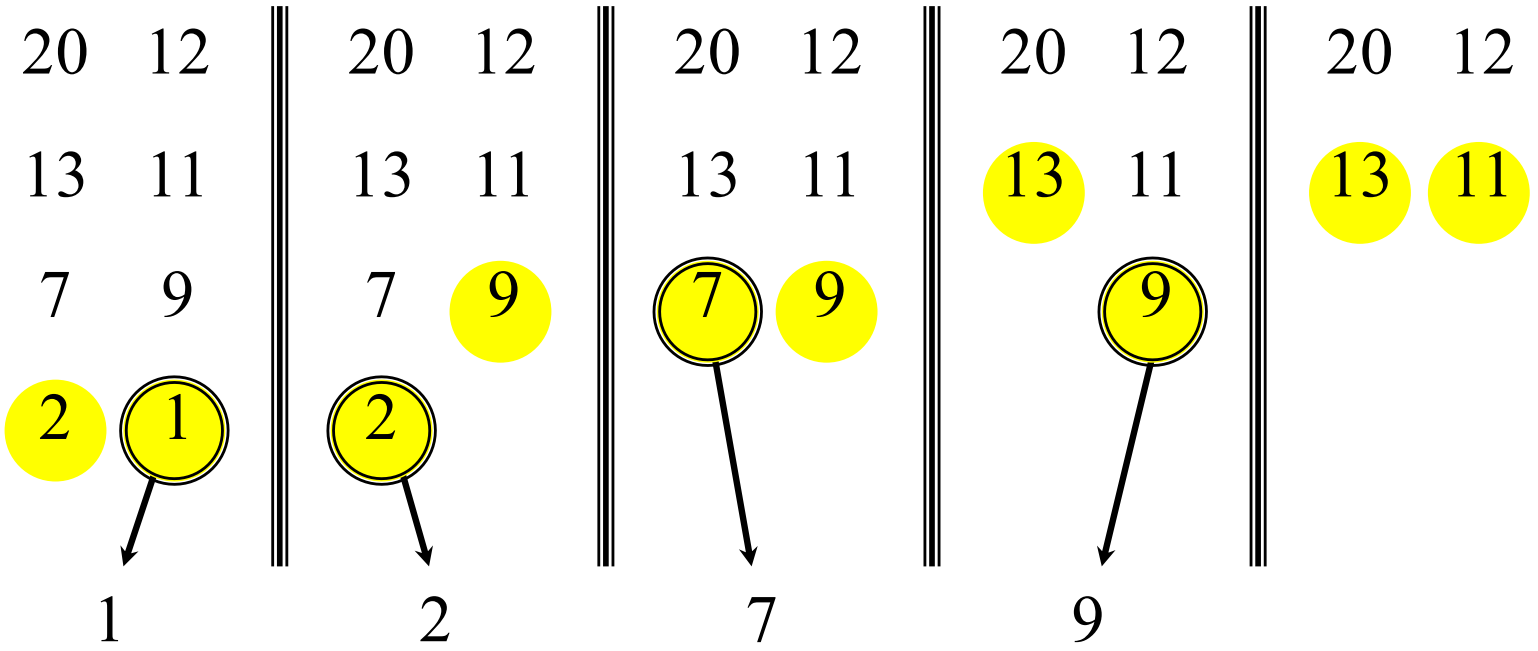
Merging two sorted arrays



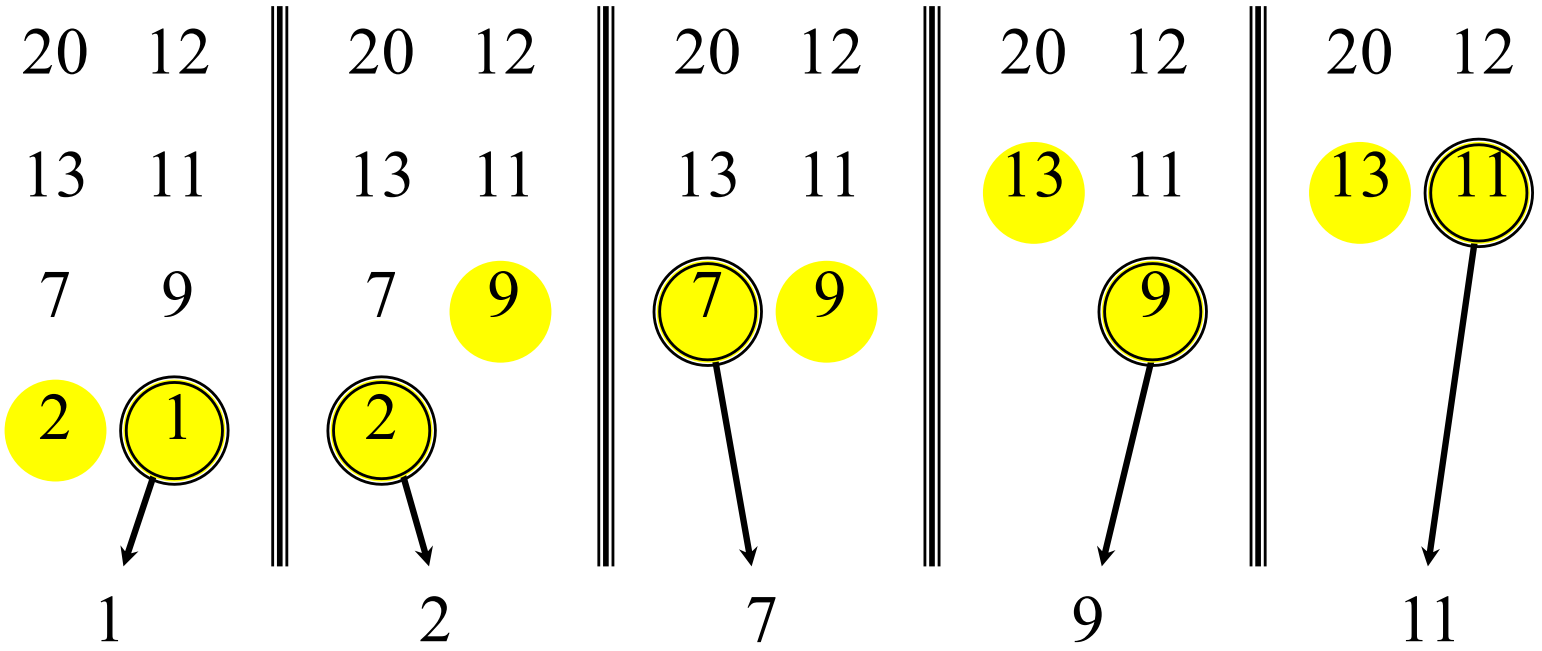
Merging two sorted arrays



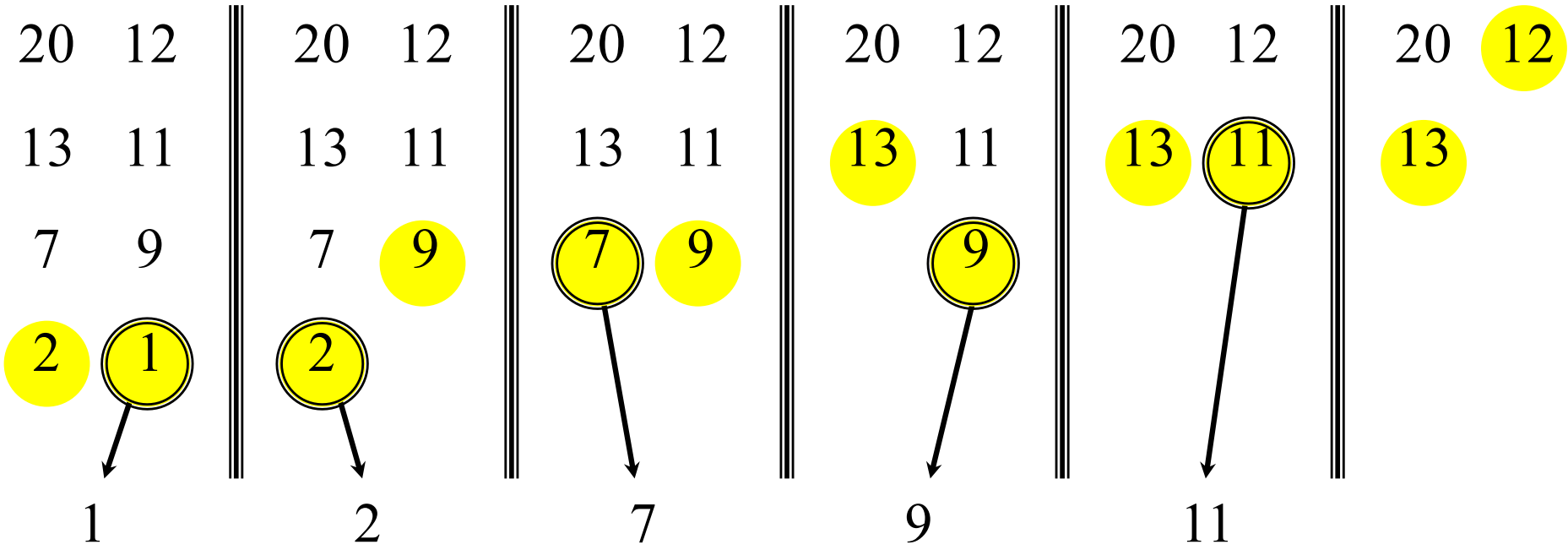
Merging two sorted arrays



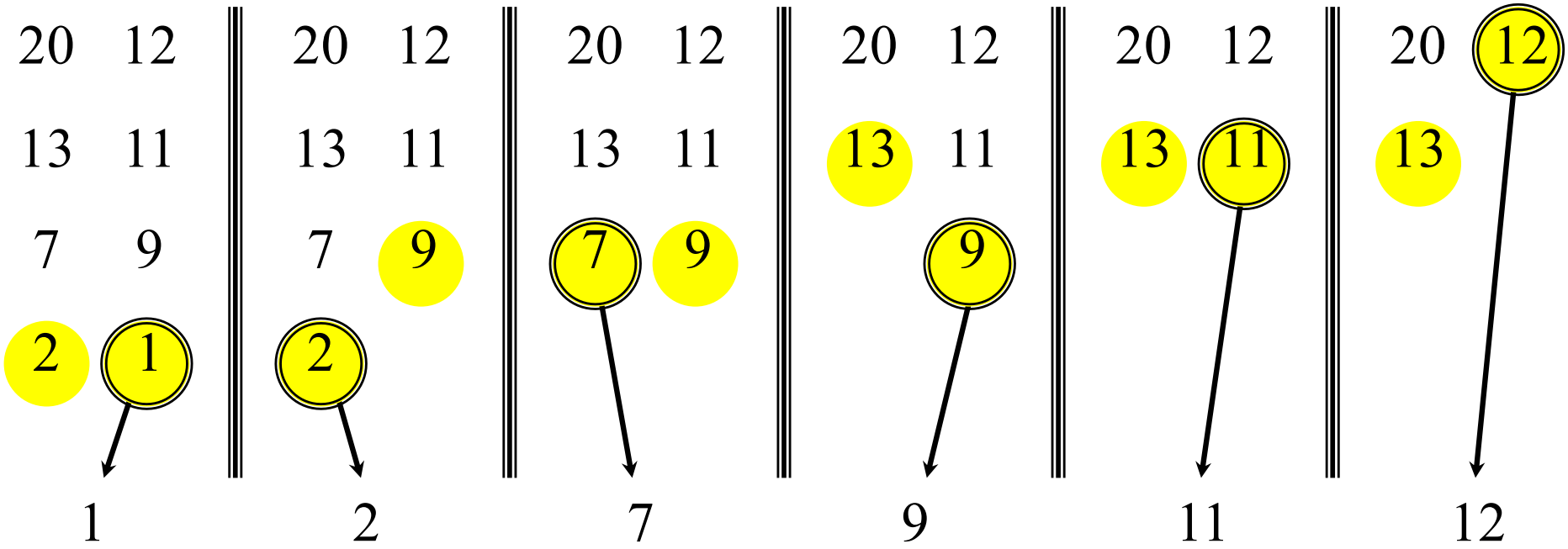
Merging two sorted arrays



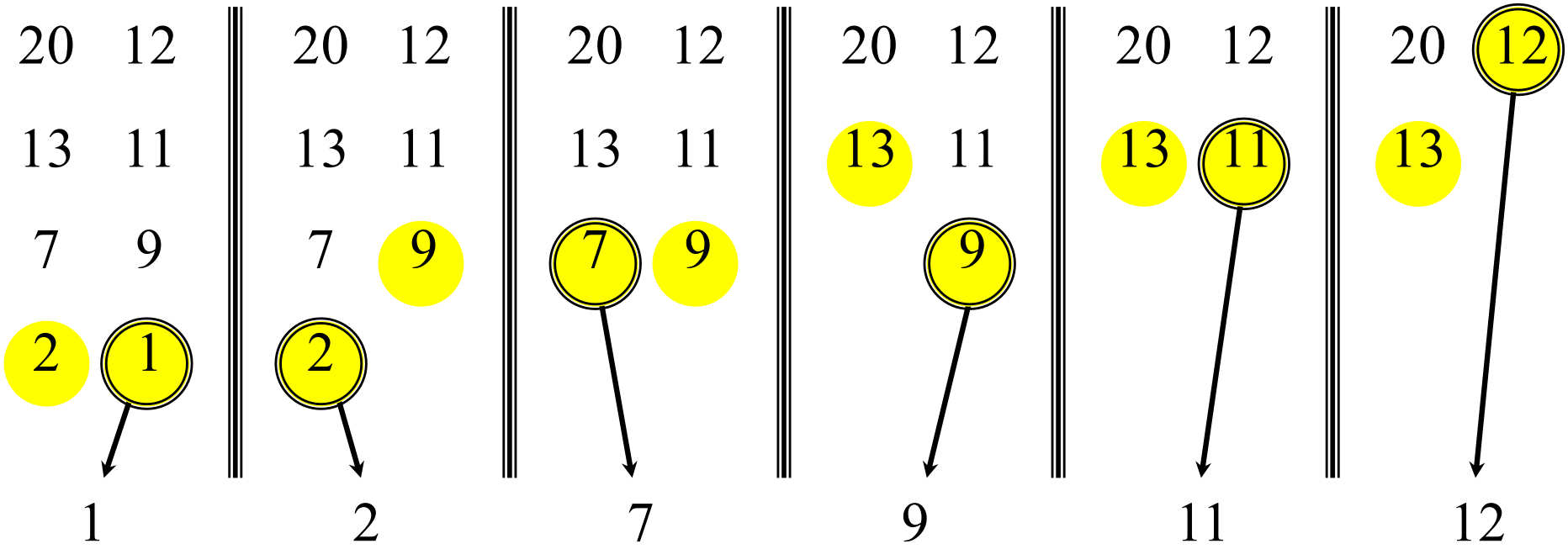
Merging two sorted arrays



Merging two sorted arrays



Merging two sorted arrays



Time = $\Theta(n)$ to merge a total of n elements (linear time).

MERGE-SORT running time

Analysis of merge sort

Here's how to set up the recurrence for $T(n)$, the worst-case running time of merge sort on n numbers.

Divide: The divide step just computes the middle of the subarray, which takes constant time. Thus, $D(n) = \Theta(1)$.

Conquer: Recursively solving two subproblems, each of size $n/2$, contributes $2T(n/2)$ to the running time (ignoring the floors and ceilings, as we discussed).

Combine: Since the MERGE procedure on an n-element subarray takes $\Theta(n)$ time, we have $C(n) = \Theta(n)$.

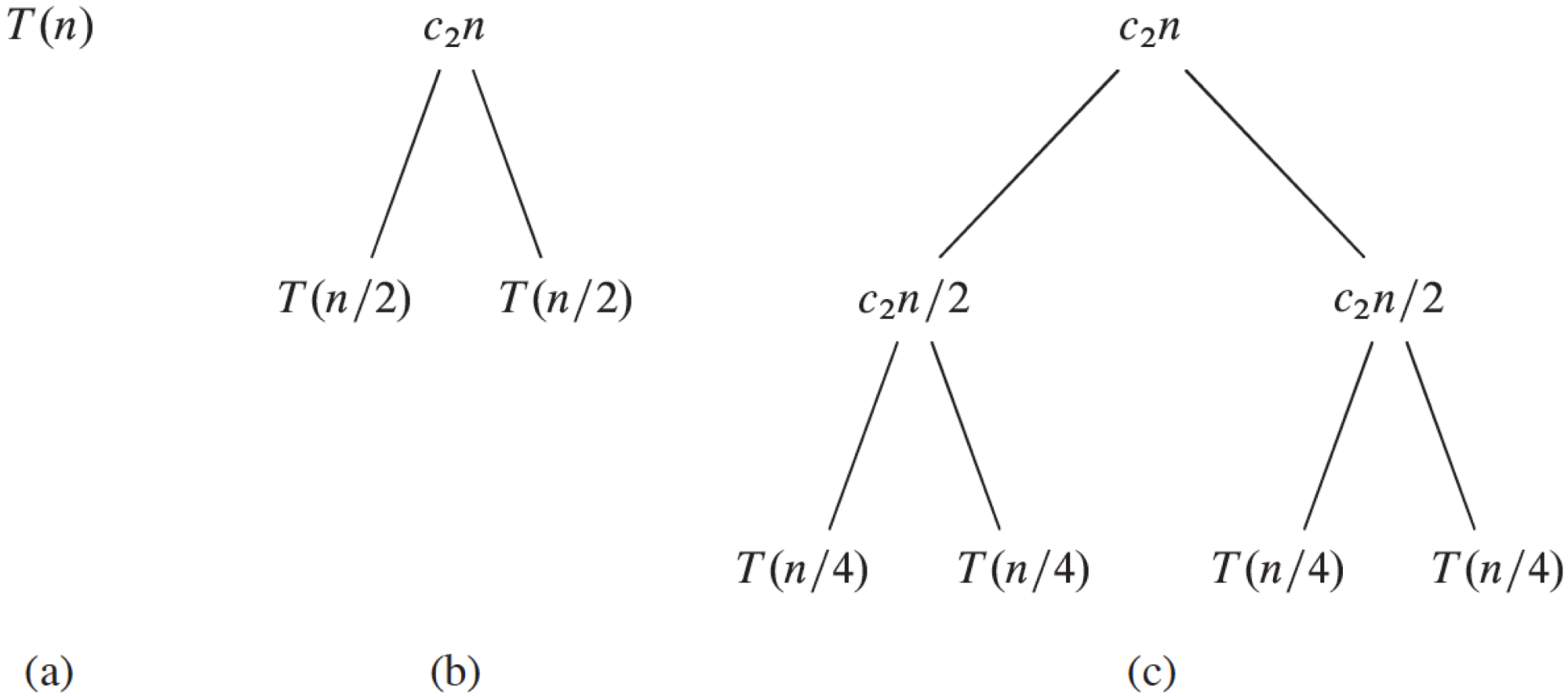
MERGE-SORT running time

$$T(n) = 2T(n/2) + \Theta(n)$$

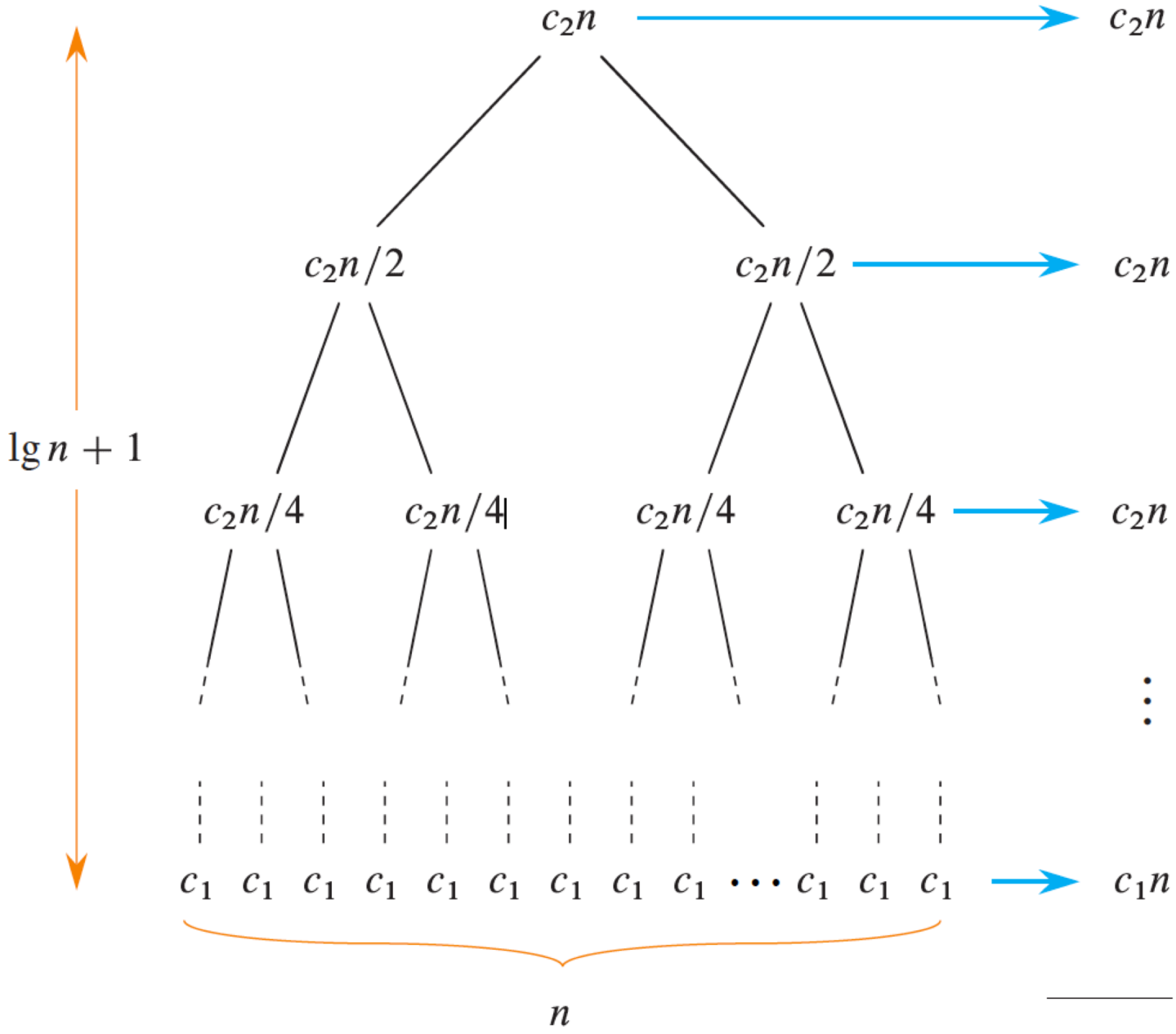
$$T(n) = \begin{cases} c_1 & \text{if } n = 1 , \\ 2T(n/2) + c_2 n & \text{if } n > 1 , \end{cases}$$

MERGE-SORT running time

Recursion tree:



MERGE-SORT running time



MERGE-SORT running time

$$c_2 n \lg n + c_1 n = \Theta(n \lg n).$$

$$T(n) = \Theta(n \lg n)$$